

Normas de codificación SYSCOOP Solution

Basdo en :

<https://comunidadvfp.blogspot.com/2016/07/normas-de-codificacion-en-visual-foxpro.html>

Aplicar buenas normas de codificación es importante para el desarrollo de cualquier proyecto; pero es particularmente importante cuando muchos desarrolladores están trabajando en el mismo proyecto. Tener establecido pautas de codificación contribuye a garantizar que el código es de alta calidad, con pocos errores y de fácil mantenimiento.

Estas normas o pautas se basan en años de desarrollo de aplicaciones con:

1. Visual Studio .Net
2. Visual FoxPro
3. Java
4. PHP Laravel
5. Otros

Y de aprendizaje de cuáles técnicas de codificación trabajan mejor. Además, muchos conceptos han sido adaptados a partir de Code Complete (Microsoft Press, ISBN 1-55615-484-4) escrito por Steve McConnell. Este libro, es considerado una de las guías más importantes en prácticas de codificación. Puede que usted no esté de acuerdo con éstas normas, no hay problemas. Para mí, estas normas han funcionado bien.

1.- Normas de codificación [Convenciones de variables.]

No utilice guiones bajos en nombres de variables. Mezclar mayúsculas y minúsculas mejora la legibilidad. La primera letra de la variable debe indicar su alcance y debe ser siempre minúscula. Trate de evitar el uso de variables públicas (PUBLIC) y privadas. En su lugar utilice propiedades de los objetos de su aplicación, propiedades de formularios y variables locales.

- l - Local
- g - Global
- p - Private
- t - Parameter

La segunda letra indica el tipo de dato.

- c - Character
- n - Numeric
- d - Date
- t - DateTime
- l - Logical
- m - Memo

- a - Array
- o - Object
- x - Indeterminate

1.1.- Ejemplos Convenciones de Variable

1.1.1.- Visual Studio .Net

```
public int setValores(string tcNombre, int tnCodigo, DateTime tdFechaNac,
                    bool tlEsValido, decimal tnValor)
{
    string lcNombre = tcNombre;
    int lnCodigoInterno = tnCodigo;
    DateTime ldFechaNacimiento = tdFechaNac;
    bool llEsValidoRegistro = tlEsValido;
    decimal lnCosto = tnValor;

    return lnCodigoInterno;
}
```

1.1.2.- Visual FoxPro

```
FUNCTION setValores(tcNombre AS String, tnCodigo AS Integer, tdFechaNac AS DateTime,;
                    tlEsValido AS Boolean, tnValor AS Numeric) AS Integer

    LOCAL lcNombre, lnCodigoInterno, ldFechaNacimiento, llEsValidoRegistro, lnCosto

    lcNombre = tcNombre
    lnCodigoInterno = tnCodigo
    ldFechaNacimiento = tdFechaNac
    llEsValidoRegistro = tlEsValido
    lnCosto = tnValor

    RETURN lnCodigoInterno;
ENDFUNC
```

1.1.3.- Java

```
public int setValores(String tcNombre, int tnCodigo, Date tdFechaNac,
                    boolean tlEsValido, double tnValor) {

    String lcNombre = tcNombre;
    int lnCodigoInterno = tnCodigo;
    Date ldFechaNacimiento = tdFechaNac;
    boolean llEsValidoRegistro = tlEsValido;
    double lnCosto = tnValor;
```

```
    return lnCodigoInterno;
}
```

1.1.4.- PHP Laravel

```
<?php
public function setValores(string $tcNombre, int $tnCodigo, string $tdFechaNac,
                           bool $tlEsValido, float $tnValor):string
{
    $lcNombre = $tcNombre;
    $lnCodigoInterno = $tnCodigo;
    $ldFechaNacimiento = $tdFechaNac;
    $llEsValidoRegistro = $tlEsValido;
    $lnCosto = $tnValor;

    return $lnCodigoInterno;
}
?>
```

1.2.- Convenciones para nombrar objetos

Las primeras tres letras del nombre de un objeto deben ser utilizadas para indicar el tipo del objeto.

- chk - Check box
- cbo - Combo box
- cmd - Command button
- cmg - Command Group
- cnt - Container
- ctl - Control
- cus - Custom
- edt - Edit box
- frm - Form
- frs - Form set
- grd - Grid
- grc - Grid Column
- grh - Grid Column Header
- img - Image
- lbl - Label
- lin - Line
- lst - List box
- olb - OLE Bound Control
- ole - OLE Object como un ActiveX Control
- opg - Option Group
- pag - Page
- pgf - Pageframe
- sep - Separator
- shp - Shape

- spn - Spinner
- txt - Text box
- tmr - Timer
- tbr - Toolbar

1.3.- Normas para el código fuente con comentarios de los metodos

1. Utilice abundantes espacios en blanco. Hará más legible su código.
2. Utilice tabuladores, en lugar de espacios para indentar.

1.3.1.- Visual Studio .Net

1.3.2.- Visual FoxPro

1.3.3.- Java

1.3.4.- PHP Laravel

1.3.4.1 Metodo(Nomenclatura)

```
/**
 * Metodo que devuelve un Saludo en base a tcMensaje y tnNivel
 * @method      HolaMundo()
 * @author      Ing. Autor
 * @fecha       01-04-2021
 * @param       \Illuminate\Http\Request $request
 * @return      string Saludo
 */
public function HolaMundo(Request $request)
{
    $tcMensaje = $request->input('tcMensaje');
    $tnNivel = $request->input('tnNivel');
    $lcSaludo = "tcMensaje = $tcMensaje tnNivel = $tnNivel";
    return response()->json($lcSaludo);
}
```

1.3.4.1 Clase(Nomenclatura)

```
<?php
/**
 *
 * Clase que representa la cabecera de un Factura Computarizada en linea
 *
 * @category    Facturas
 * @package     CXML
 * @author      Ing. Alfonso Salgado Flores
 * @copyright   2000-2030 SYSCOOP Solution
 * @license     Pay
 * @version     Release: @package_version@
 * @todo        Parte de la facturacion Computarizada en Linea
 * @fecha       16-06-2021
 */
class XMLFacturaComputarizadaCompraVenta
{
    private array $aOpciones;

    function __construct()
    {
        $this->aOpciones = array();
    }

    /**
     * Serializa la Factura(objeto) CompraVenta en formato XML
     * @author    Ing. Alfonso Salgado Flores
     * @param     $toFCCompraVenta FacturaComputarizadaCompraVenta
     * @param     $toFCCompraVentaDetalle FacturaComputarizadaCompraVentaDetalle
     * @return    string FacturaCompraVenta Serializada en formato XML
     */
    public function serializarFactura($toFCCompraVenta, $toFCCompraVentaDetalle):string
    {

    }

}

?>
```